PRT565

Machine Learning, Artificial Intelligence and Algorithms

Assessment 4 — Group Report

Comparing Machine Learning Models for IoT Network Traffic

Group 16 | Darwin Campus

Raman Lamichhane (s401547)

Saroj Dhakal (s399846)

Rejan Sapkota (s400142)

Report date: 15 October 2026

Main report: 3,500 words (prose only; excludes cover, tables, captions, references and appendices)

Group Contributions

The report is divided into three parts according to the agreed allocation below. Word counts include body prose only. This table identifies responsibility for reviewing and finalising the report sections; it does not independently establish who ran the earlier experiments.

Member and ID	Report responsibility	Words
Raman Lamichhane s401547	Part 1: Problem, dataset analysis and preprocessing	1071
Saroj Dhakal s399846	Part 2: Algorithms, implementation and feature experiments	1080
Rejan Sapkota s400142	Part 3: Evaluation, results and critical understanding	1349

Each member should review their assigned section and the complete report before submission. The assignment requires every group member to submit the same ZIP separately through Learnline.

Part 1: Dataset and Preprocessing — Raman Lamichhane

Part 2: Models and Methods — Saroj Dhakal

Part 3: Results and Discussion — Rejan Sapkota

References

Data Details

Report Sources

The report expands Assessment 3 using the supplied Model_Comparison.ipynb, its embedded outputs and figures, the supplied dataset.csv, the course slides and the Assessment 4 instructions. Row and class counts were independently audited. Historical model scores are reproduced without claiming a new training run.

Dataset and Preprocessing

Member: Raman Lamichhane (s401547)

Problem and Motivation

Internet of Things devices generate network traffic while exchanging sensor readings, responding to requests and communicating with services. Attack tools also generate traffic, sometimes with distinctive packet counts, timing or protocol patterns. This project investigates whether supervised machine learning can distinguish the traffic categories represented in RT-IoT2022. The research question is: how do Decision Tree, Random Forest, Gaussian Naive Bayes and a multilayer artificial neural network compare when classifying the dataset's twelve traffic labels?

The motivation is to support network monitoring through automatic classification of completed traffic flows. A model could help an analyst identify records requiring investigation, but an incorrect classification can either create unnecessary work or hide suspicious activity. The project therefore examines class-level behaviour alongside overall accuracy. Its purpose is a controlled comparison of four learning approaches on the same data partitions, rather than a claim that a deployable intrusion detection system has been validated.

The comparison applies course concepts in preprocessing, supervised learning, feature engineering and evaluation to an imbalanced dataset.

Dataset Description

RT-IoT2022 is available through the UCI Machine Learning Repository. The accompanying research describes normal device activity and simulated attacks recorded in an IoT environment, with bidirectional flow attributes obtained using Zeek and Flowmeter (Sharmila & Nagapadma, 2023). The supplied CSV contains 123,117 rows and 85 columns: 83 input predictors, the target Attack_type and an exported row index. The index is excluded because it records file position rather than a meaningful traffic characteristic.

Predictors include ports, protocol, service, duration, packet statistics, payload summaries, flags and timing. Protocol and service are categorical; the other 81 predictors are numeric. All twelve supplied labels are preserved, including ARP_poisioning.

An independent audit of the supplied CSV confirms 5,195 exact duplicate records after excluding the exported index. Removing them leaves 117,922 records. The earlier Assessment 3 statement that 90,089 records remained was incorrect: 90,089 is the count of the largest class, DOS_SYN_Hping. The alternative total of 117,923 was also a counting error. These corrections matter because class proportions, split sizes and the interpretation of accuracy depend on a consistent denominator.

Table 1. Deduplicated dataset class distribution.

Traffic class	Records	Share (%)
DOS_SYN_Hping	90,089	76.397
Thing_Speak	7,654	6.491
ARP_poisioning	7,625	6.466
MQTT_Publish	4,142	3.512
NMAP_UDP_SCAN	2,584	2.191
NMAP_XMAS_TREE_SCAN	2,010	1.705
NMAP_OS_DETECTION	2,000	1.696
NMAP_TCP_scan	1,002	0.850
DDOS_Slowloris	533	0.452
Wipro_bulb	219	0.186
Metasploit_Brute_Force_SSH	36	0.031
NMAP_FIN_SCAN	28	0.024

Dataset Analysis

The distribution is highly uneven. DOS_SYN_Hping contributes 76.40% of the deduplicated records, whereas NMAP_FIN_SCAN has only 28 examples. A majority predictor can therefore obtain apparently impressive accuracy without distinguishing most categories. Conversely, rare-class estimates can change sharply when only one or two predictions change. This distribution motivates macro-averaged evaluation, stratification and explicit reporting of class support (He & Garcia, 2009).

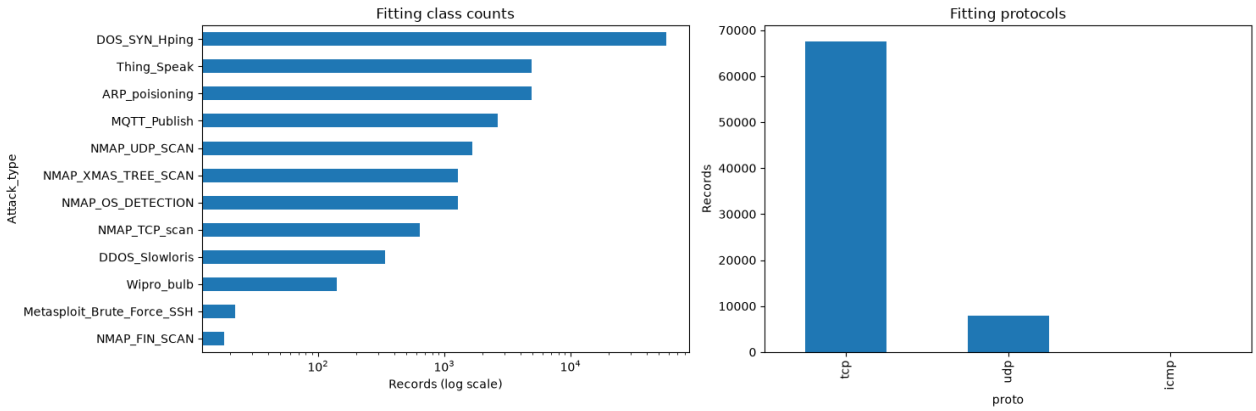


Figure 1. Class counts and protocol frequencies in the fitting partition. Source: supplied Model_Comparison.ipynb.

Figure 1 uses a logarithmic count axis for the classes so that small categories remain visible. The important observation is not simply that the dataset contains many attack records; it is that one particular attack category dominates. Protocol frequencies also show that input categories are unevenly represented. These charts justify checking class-specific performance rather than treating every correct prediction as equally informative about the breadth of classification ability.

The saved fitting-data summary gives a mean flow duration of approximately 4.03 dataset units, a median of 0.000004, and a maximum of 21,728.34. The large difference between the mean and median indicates a heavily skewed distribution. Forward packet totals similarly have a median of one and a maximum of 4,345. A small number of large flows therefore occupy much of the numeric range, making raw scale a potentially important modelling issue.

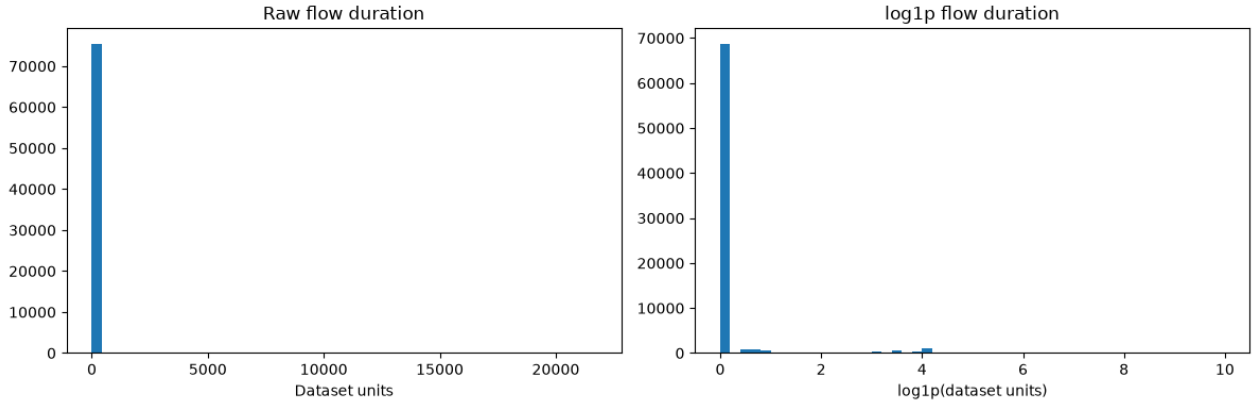


Figure 2. Raw and log-transformed flow duration in the fitting partition. Source: supplied Model_Comparison.ipynb.

Figure 2 shows why logarithmic transformation was tested. In the raw histogram, large values compress most observations near the lower end of the axis. The transformed view spreads smaller values and makes the distribution easier to inspect. Transformation changes the representation, not the number of observations or their labels. It should therefore be evaluated empirically rather than assumed to improve every algorithm.

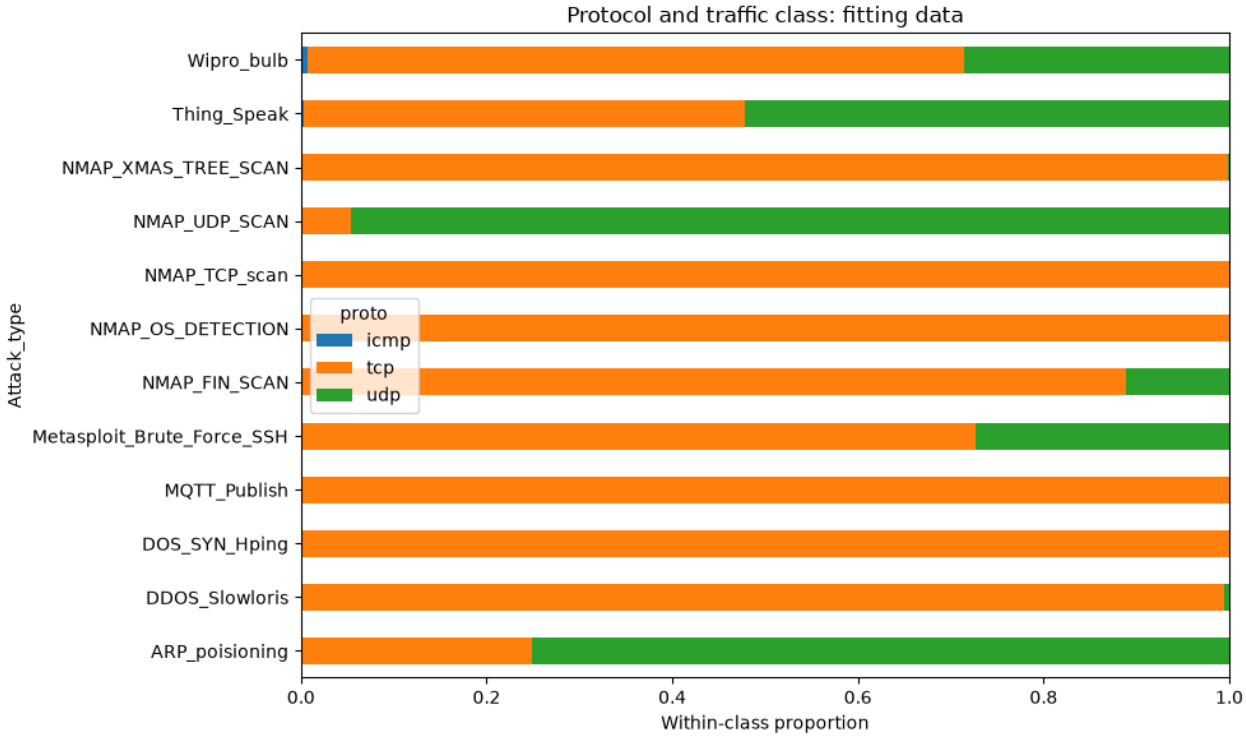


Figure 3. Within-class protocol proportions in the fitting partition. Source: supplied Model_Comparison.ipynb.

Figure 3 shows that protocol composition differs between traffic categories. Protocol therefore contributes predictive information but cannot uniquely identify an attack. Other flow attributes remain necessary, and laboratory protocol patterns may differ from operational networks.

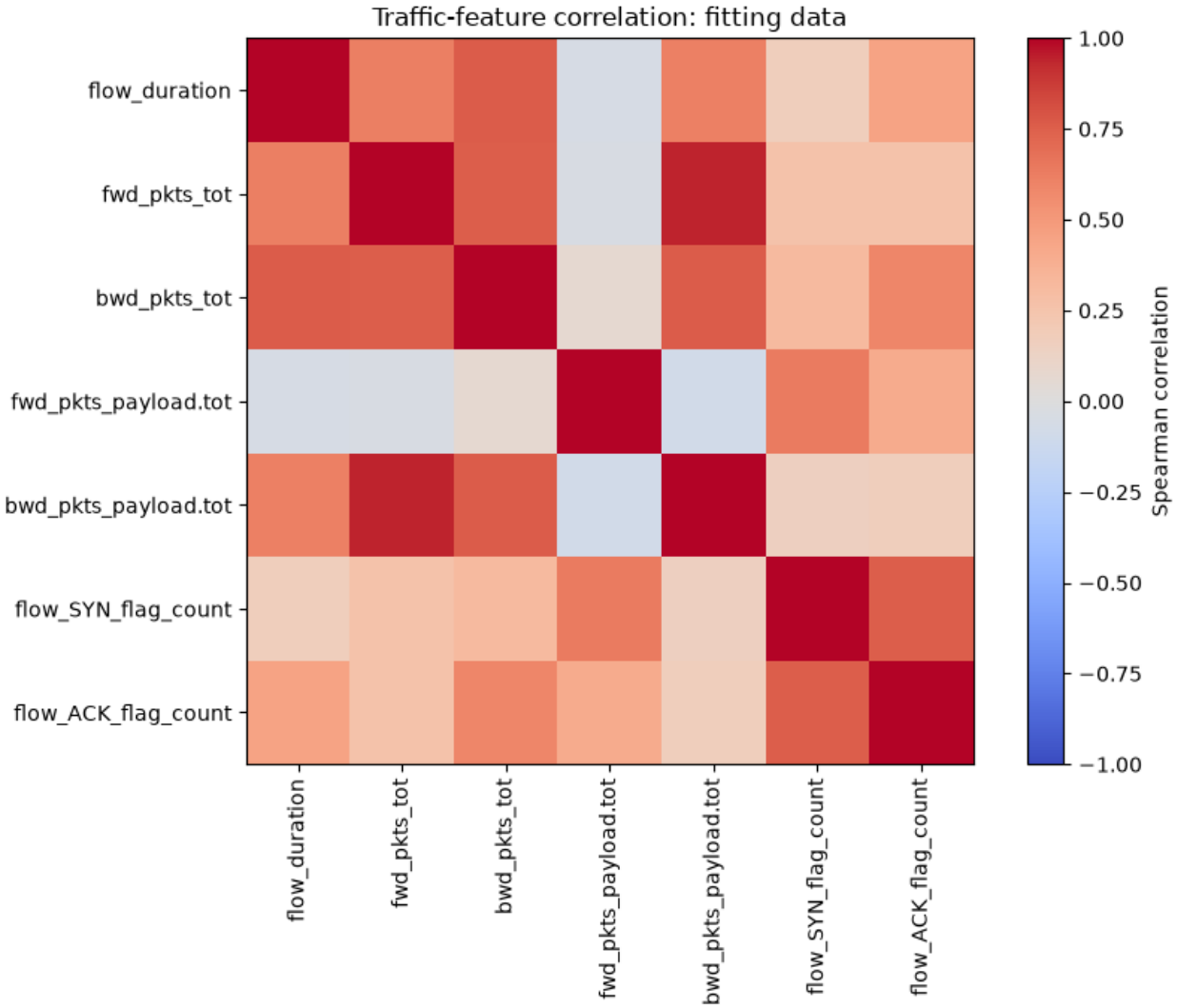


Figure 4. Spearman correlations among selected traffic predictors. Source: supplied Model_Comparison.ipynb.

Figure 4 examines relationships between duration, directional packet counts, directional payload totals and flag counts. Related traffic summaries need not provide independent information. Correlation can explain why several predictors appear useful together and why the independence assumption of Naive Bayes is questionable. However, a pairwise correlation plot cannot establish causation or identify all nonlinear relationships. It is an exploratory tool, not sufficient evidence for automatically deleting a feature.

Data Preprocessing

The pipeline removes the exported index and exact duplicates before splitting. The notebook also reports six groups of identical predictor vectors with conflicting target labels. These records are retained, while matching predictor vectors are kept in the same partition. Otherwise, identical inputs could appear in both training and testing and make the evaluation easier. Conflicting labels also indicate that the available predictors cannot perfectly distinguish every record.

The supplied data have no missing values in the audit. Nevertheless, numeric median imputation and categorical constant imputation are included in the pipeline for consistent handling of missing inputs. Infinite numeric values are replaced with missing values before imputation. These safeguards do not imply that missing observations were found or repaired in this particular file. The distinction between observed data quality and precautionary pipeline design is maintained throughout the report.

Numeric predictors are standardised using StandardScaler. Categorical predictors are one-hot encoded, with unknown categories ignored by the encoder during transformation. The original 83 predictors become 94 encoded inputs in the saved experiment. Scaling and imputation are fitted on the fitting partition during selection, and then applied to validation without refitting. Fitting preprocessing within the training partition prevents validation information from entering estimated parameters, supporting valid model assessment (Kaufman et al., 2012).

Two numeric representations are compared: the original values and a signed logarithmic transformation, $\text{sign}(x) \times \log(1 + |x|)$. The signed form supports negative values without requiring their removal. It is applied to all numeric predictors, including ports; this is a broad experimental transformation rather than a claim that each feature has the same physical meaning. Standardisation follows transformation. Both representations use the same partitions so that changes in validation performance can be compared fairly.

The notebook uses the first fold of a seeded five-fold StratifiedGroupKFold split for testing, then repeats the procedure within development data for validation. It does not report an average across five folds. The resulting partitions contain 75,469 fitting records, 18,868 validation records and 23,585 test records, approximately 64%, 16% and 20%. All twelve classes appear in each partition. Identical predictor groups do not cross partitions, although independent devices or sessions cannot be guaranteed.

Models and Methods

Member: Saroj Dhakal (s399846)

Model Selection

All four algorithms solve the same multiclass task and use the same fitting, validation and test partitions. Each algorithm's settings are selected using validation macro F1. The selected models are subsequently refitted using the combined fitting and validation records before final testing. This distinction separates parameter selection from performance assessment. The test partition is used to compare the frozen choices, rather than to choose another setting after observing a favourable result.

Decision Tree

A Decision Tree divides records using conditions on individual predictors. At each internal node, a threshold partitions the current observations; a leaf assigns a class using the training records reaching that leaf. The resulting rules can represent nonlinear interactions, such as a port condition followed by a packet-rate threshold. Recursive partitioning and control of model complexity underpin tree learning (Hastie et al., 2009). The implementation uses scikit-learn's default Gini criterion rather than entropy.

Maximum depths of ten and unrestricted depth are compared with minimum leaf sizes of one and five. Larger leaves may reduce overfitting but obscure rare patterns.

The selected tree uses original numeric features, unrestricted depth and minimum leaf size one, achieving validation macro F1 of 0.977105. The depth-ten alternative reaches 0.976193. This close ordering does not establish that unrestricted depth is generally optimal.

Random Forest

Random Forest combines trees trained with bootstrap sampling and random feature subsets, reducing dependence on a single tree's particular rules (Breiman, 2001). This is useful when several traffic predictors contain overlapping information. Different trees can rely on different combinations of timing, payload and protocol features, and their combined predictions can be more stable than those of an individual tree. The method also produces impurity-based feature importance values for exploratory inspection.

The experiment fixes the forest size at 100 trees and compares maximum depths of sixteen and unrestricted depth, with minimum leaf sizes of one and five. The selected forest uses signed-log numeric features, maximum depth sixteen and minimum leaf size one. Its validation macro F1 is 0.973912. The best forest on original numeric values obtains 0.972264. This small improvement suggests that transformation had limited benefit for the forest in this experiment.

Trees use feature thresholds rather than distances, so standardisation is not essential. Shared preprocessing simplifies comparison. Small raw-versus-log differences can reflect numerical

precision and split behaviour rather than fundamentally improved tree expressiveness.

Naive Bayes

Gaussian Naive Bayes predicts a class using its prior probability and estimated feature likelihoods. Its simplifying assumption is that predictors are conditionally independent given the class, with numeric likelihoods represented through Gaussian distributions. Naive Bayes is a probabilistic supervised method based on class-conditional distributions (Domingos & Pazzani, 1997). Here it offers a comparatively simple baseline against models that can learn more complicated interactions.

Correlated counts, skewed distributions and binary encoded categories challenge the Gaussian independence assumptions. The fitted probability model is therefore an approximation.

The tested variance smoothing values are 0.000000001, 0.00001 and 0.01. Smoothing stabilises variance estimates and prevents extremely small estimated variances from dominating the likelihood calculation. The selected model uses logarithmic numeric features and smoothing of 0.01. Its validation macro F1 is 0.675955, compared with 0.596936 for the best original-feature configuration. The transformation therefore benefits this model substantially, although its selected validation performance remains well below the tree models.

Neural Network

The neural network is a feedforward multilayer perceptron, abbreviated MLP in the notebook. It receives the encoded predictor vector, passes it through two dense hidden layers, and outputs twelve softmax scores. Hidden layers use ReLU activation and each is followed by dropout at a rate of 0.25. The network learns feature combinations through weight updates. Dropout reduces overfitting by randomly omitting units during training (Srivastava et al., 2014).

Hidden widths of (32,16) and (64,32) are tested with both representations. Training uses Adam (Kingma & Ba, 2015), learning rate 0.001, sparse categorical cross-entropy and batches of 256. Integer labels are consistently encoded; the largest softmax output determines the prediction. Calibration is not evaluated.

Training allows up to forty epochs. Early stopping monitors validation loss with patience of five and restores the weights from the best validation-loss epoch. Validation-based early stopping limits overfitting during iterative training (Prechelt, 1998). Importantly, the criterion for stopping an individual training run is validation loss, while the criterion for comparing configurations is validation macro F1. These objectives are related but not identical, particularly for an imbalanced dataset.

The selected network has hidden widths of 64 and 32, logarithmic numeric inputs and validation macro F1 of 0.961065. Its best validation-loss epoch is thirty. Final refitting uses the combined development partition for that fixed number of epochs, without using test observations for stopping. The smaller original-feature network reaches only 0.762620 validation macro F1. The

larger architecture and transformed representation are therefore materially stronger within the four configurations evaluated.

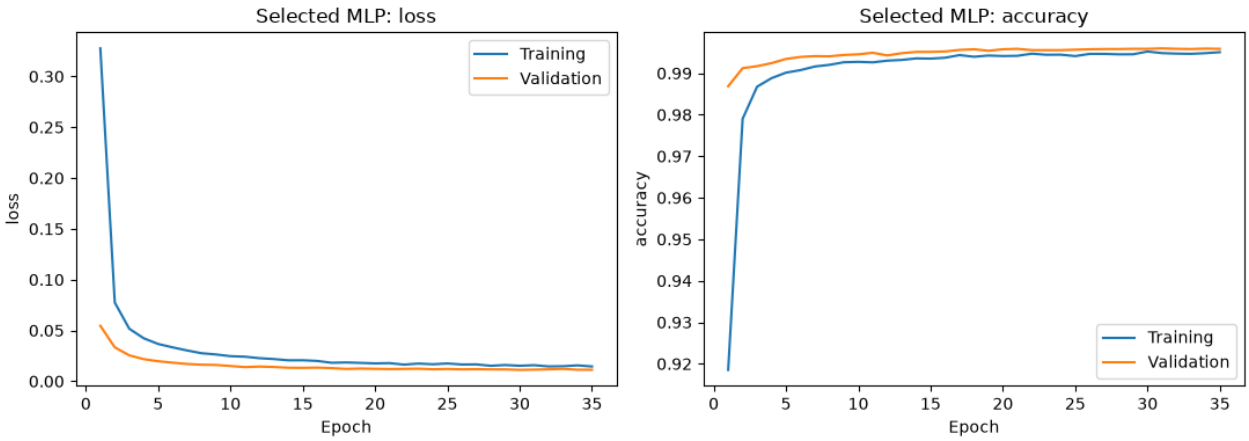


Figure 5. Training and validation curves for the selected neural network configuration. Source: supplied Model_Comparison.ipynb.

Figure 5 helps inspect convergence and differences between training and validation. However, high accuracy can hide poor rare-class performance. Learning curves therefore require interpretation alongside macro F1 and class recall.

Feature Selection and Engineering

The notebook includes supplementary validation-only experiments after the primary comparison. A forest-based selector chooses the top twenty encoded features using fitting data alone. The selected forest is then trained on those inputs and assessed on validation data. Its macro F1 drops from 0.973912 with all 94 inputs to 0.806564 with twenty. This substantial decline shows that aggressive dimensionality reduction sacrifices useful information in this particular setup.

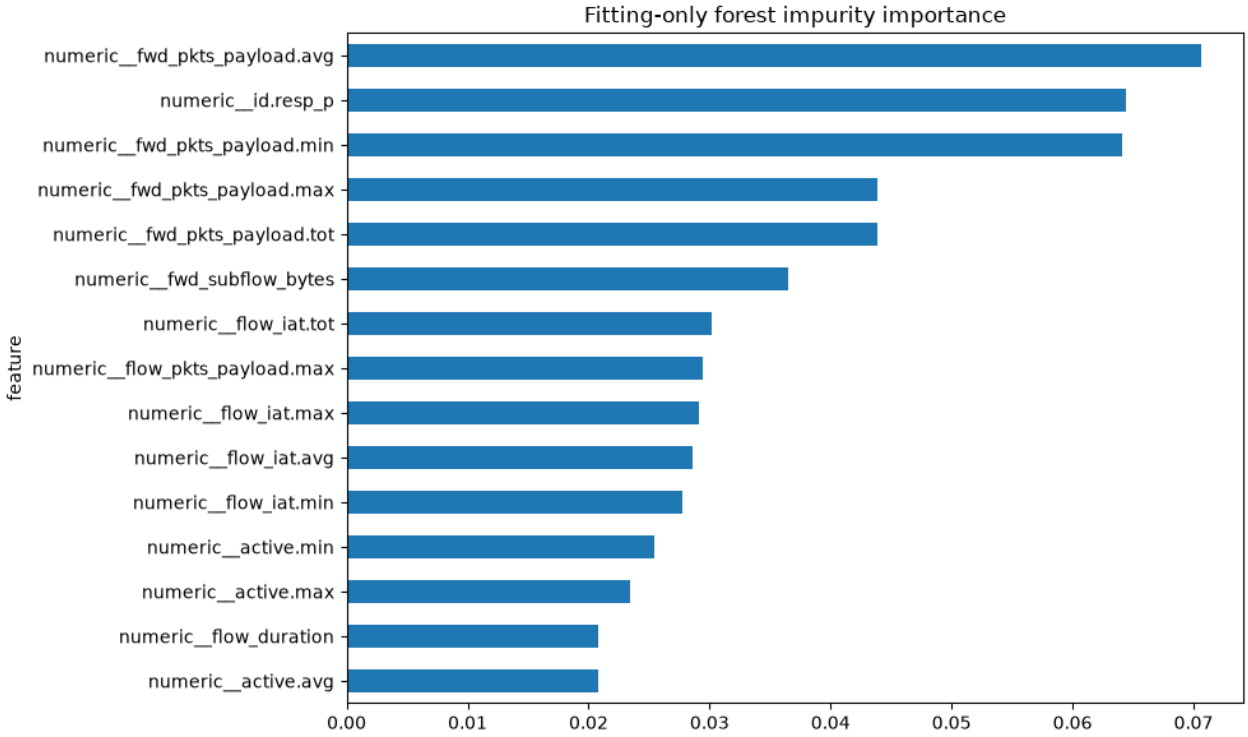


Figure 6. Leading impurity-based feature importance values from the fitting-data forest. Source: supplied Model_Comparison.ipynb.

Figure 6 highlights forward payload statistics and destination port. Impurity importance indicates predictive associations, not causation. Correlated features can share importance, while predictors with many split points can be favoured (Strobl et al., 2007). Port importance suggests possible collection-specific patterns.

Two engineered ratios are additionally tested. Packet-direction balance is (forward packets − backward packets)/(forward packets + backward packets + 1), and backward-payload share is backward payload/(forward payload + backward payload + 1). The added constant prevents division by zero. Both ratios are computed from the original counts before being included in numeric preprocessing, alongside the selected logarithmic original predictors. They summarise traffic direction in a way that absolute counts do not express directly.

Adding the two ratios produces 96 encoded inputs and validation macro F1 of 0.974771, an increase of approximately 0.086 percentage points over the full-feature forest. This is a small improvement from one validation partition, rather than convincing evidence of a general advantage. These supplementary results do not replace the previously selected primary models and are not reported as independent test improvements. A future experiment would need a fresh evaluation design to assess their benefit reliably.

Results and Discussion

Member: Rejan Sapkota (s400142)

Evaluation Measures

Evaluation must match the purpose of distinguishing twelve unequal traffic categories. Accuracy is the number of correctly classified records divided by the total number of evaluated records. It provides a direct measure of overall correctness, but each record contributes equally, so the largest class dominates the result. In this dataset, high accuracy can coexist with poor detection of several smaller categories. Accuracy is therefore useful, but insufficient as the only basis for comparison.

For each class, precision is TP/(TP + FP), recall is TP/(TP + FN), and F1 is 2 × precision × recall/(precision + recall). Here TP means records correctly assigned to that class, FP means other records incorrectly assigned to it, and FN means its records assigned elsewhere. Precision measures the reliability of a predicted class, while recall measures how much of the actual class is recovered. F1 balances these two dimensions.

The reported precision, recall and F1 values are macro averages across all twelve classes. Each class contributes equally, irrespective of its support. Macro F1 is the average of the individual class F1 scores; it is not generally the harmonic mean of macro precision and macro recall. The implementation sets undefined metric divisions to zero. Classification metrics and their averaging conventions require interpretation in relation to task structure and class distributions (Sokolova & Lapalme, 2009).

In this multiclass task, a false negative for one attack may be a prediction of another attack. Class recall therefore does not directly measure intrusion detection. Operational evaluation must define harmful errors and calculate security measures accordingly.

Test Results

The final test partition contains 23,585 records. All selected configurations are refitted on the combined development data before their predictions are compared on that same test partition. Table 2 reproduces the rounded metrics saved in the supplied notebook. These are recorded experiment outputs; this report does not claim that every model has been retrained in the present environment. The dataset audit was independently checked while preparing the report.

Table 2. Saved final test metrics; percentages rounded to two decimal places.

Model	Accuracy	Macro precision	Macro recall	Macro F1
Decision Tree	99.79%	97.27%	96.83%	97.05%
Random Forest	99.84%	98.91%	97.08%	97.84%

Model	Accuracy	Macro precision	Macro recall	Macro F1
Naive Bayes	92.92%	64.55%	83.68%	65.19%
Multilayer ANN	99.59%	98.60%	93.85%	95.93%
Majority baseline	76.39%	6.37%	8.33%	7.22%

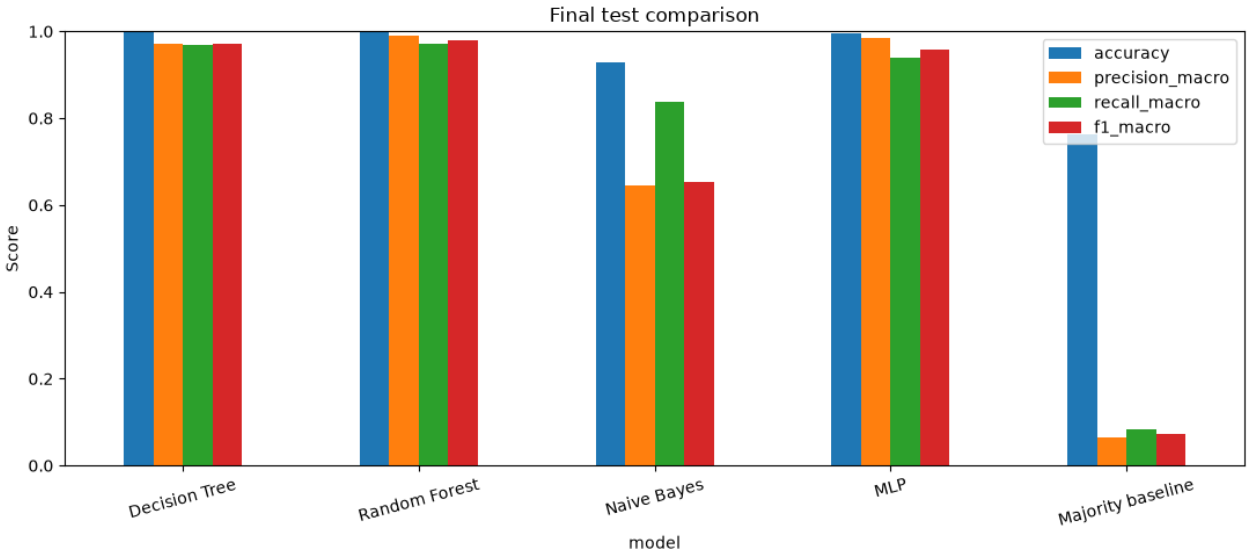


Figure 7. Final test accuracy and macro-averaged precision, recall and F1. Source: supplied Model_Comparison.ipynb.

The majority baseline always predicts DOS_SYN_Hping, achieving 76.39% accuracy but only 7.22% macro F1 and 8.33% macro recall. Recovering only one of twelve classes demonstrates why accuracy requires balanced metrics. All learned models improve on it.

Random Forest achieves the highest observed test accuracy, 99.84%, and macro F1, 97.84%. Decision Tree follows with 99.79% accuracy and 97.05% macro F1. The neural network records 99.59% accuracy and 95.93% macro F1. Naive Bayes reaches 92.92% accuracy but only 65.19% macro F1. Figure 7 makes the gap between Naive Bayes accuracy and balanced class performance clear, while also showing the strong overall results of the other approaches.

Model Recommendation

The Decision Tree is the model selected by the established validation rule. Its validation macro F1 of 97.7105% exceeds Random Forest's 97.3912%, the neural network's 96.1065% and Naive Bayes' 67.5955%. Random Forest obtains the strongest test metrics, but replacing the recommendation because of that result would use test information for model selection. The report distinguishes the validation-selected model from the highest-scoring tested model.

Different partition records and rare-class errors can change model ordering. The tree leads validation macro F1 by only 0.319 percentage points; repeated evaluation was not conducted. Both tree approaches perform strongly, but their relative advantage remains uncertain.

Class Performance

Aggregate scores conceal important variation. For Metasploit_Brute_Force_SSH, the test partition contains only eight records. Decision Tree, Random Forest and the neural network each achieve recall of 0.75, meaning six are correctly classified and two are assigned elsewhere. Decision Tree's precision is also 0.75, whereas Random Forest and the neural network achieve precision of 1.00 for this class. Their stronger precision explains their higher class F1 despite identical recall.

NMAP_FIN_SCAN has six test records. Decision Tree, Random Forest and the neural network classify all six correctly. This produces perfect observed class scores, but the evidence remains limited to six examples. A single additional missed record would reduce recall to approximately 83.33%. Perfect performance on a tiny sample should therefore be described as an observed result with considerable uncertainty, rather than a guarantee about future FIN scans.

The neural network's weaker macro recall is visible in less frequent categories. Its DDOS_Slowloris recall is 82.08%, compared with 98.11% for both tree models. For Wipro_bulb, it correctly recovers 75.00%, whereas Decision Tree reaches 93.18% and Random Forest 95.45%. These class differences explain why the network's very high overall accuracy does not make it the strongest balanced classifier. More complexity does not automatically improve every class.

Naive Bayes illustrates a different problem: several classes have high recall but low precision. For DDOS_Slowloris, recall is 98.11% while precision is only 15.38%. For NMAP_FIN_SCAN, recall reaches 100% but precision is 16.22%. Many records from other categories are therefore assigned these labels. Such predictions could create substantial investigation workload if interpreted as alerts, although actual alert costs were not measured in this project.

Naive Bayes recall is 34.89% for ARP_poisioning and 60.16% for Thing_Speak, despite perfect dominant-class recall. Its 92.92% accuracy therefore conceals weaker balanced performance. Distributional assumption mismatch is a plausible explanation rather than an experimentally established cause.

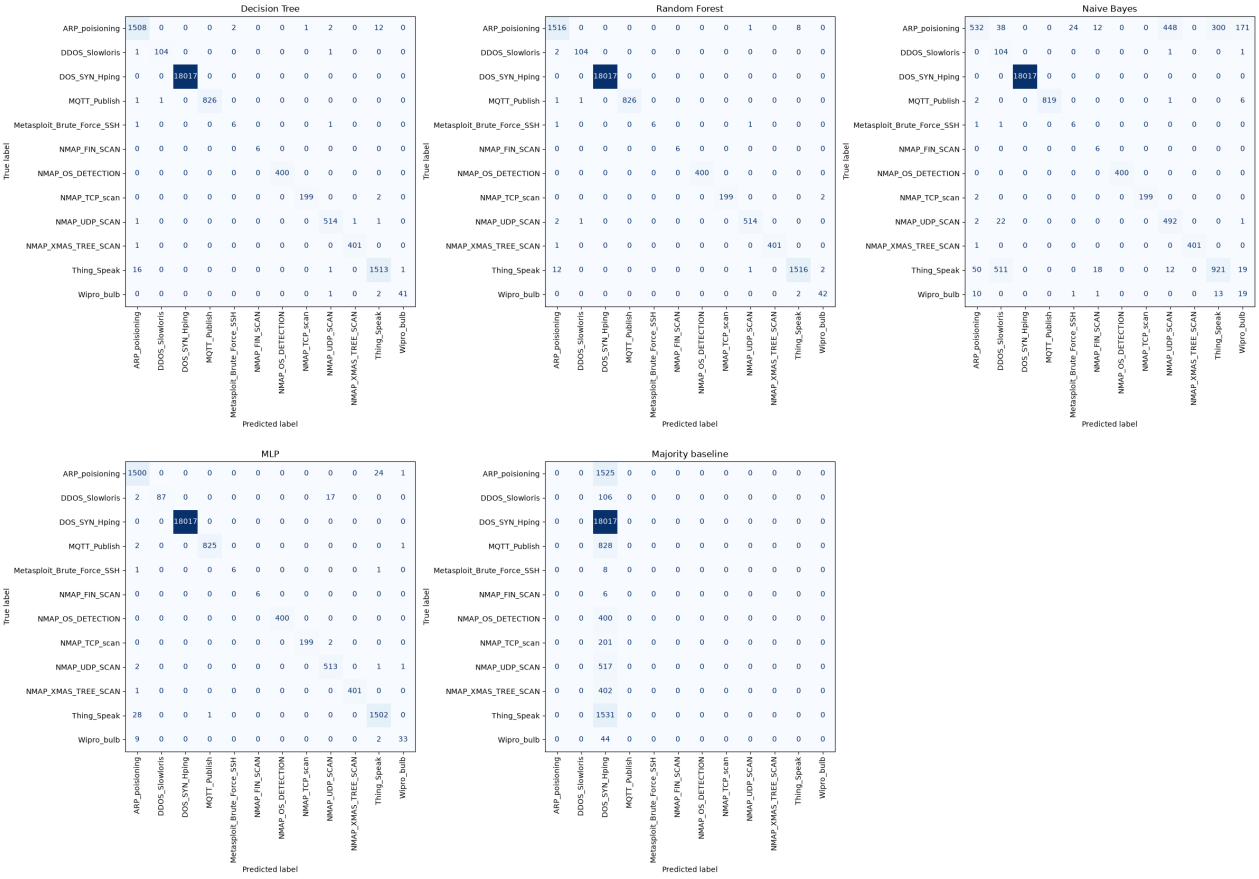


Figure 8. Saved test confusion matrices for the four models and majority baseline. Source: supplied Model_Comparison.ipynb.

Figure 8 shows broad diagonal structure for learned models and one prediction column for the baseline. Dominant-class counts can obscure small errors, making class-level reports essential. Off-diagonal entries identify misclassifications rather than automatically indicating missed attacks.

Feature Experiments and Training Time

The validation representation comparison shows that transformation has model-dependent effects. Naive Bayes improves from 59.69% to 67.60% macro F1, and the best neural network improves from 94.99% to 96.11%. Random Forest changes from 97.23% to 97.39%, while Decision Tree's best original-feature result exceeds its best transformed-feature result. These results support evaluating preprocessing as part of model selection rather than choosing one transformation solely because a histogram appears less skewed.

Selecting twenty features loses 16.73 validation macro F1 percentage points, whereas directional ratios add little. Less prominent predictors may retain useful joint information. Neither experiment establishes the optimal representation or confirms transfer to another network.

Saved refitting times are approximately 1.45 seconds for Decision Tree, 1.48 for Random Forest, 0.92 for Naive Bayes and 20.49 for the network. Hardware and implementation affect

these observations. Prediction latency, memory and total tuning cost were not measured.

Limitations

The experiment is limited to a single collection environment and one retained test partition. Matching predictor groups are separated, but correlated flows from the same device or session may still appear across partitions. No trustworthy device-held-out or temporal evaluation is demonstrated. Ports and service patterns may encode laboratory conditions, and some predictors require a completed flow. Accordingly, the results support offline classification within this dataset, not real-time detection speed or reliable transfer to unfamiliar networks.

The supplied CSV and notebook hashes differ because of line endings. Converting the supplied file to Windows line endings reproduces the saved hash exactly. The underlying dataset contents match; model scores remain the previously saved notebook outputs.

The notebook records seeds, versions and export commands. However, the supplied ZIP contains only the notebook and CSV, with embedded figures and outputs; separate trained models and prediction exports are absent. Historical package versions were not independently verified.

Future work should test independent networks, repeated grouped splits and additional rare examples. Port ablation and attack-versus-normal evaluation would clarify practical usefulness. Deployment needs latency and false-alert analysis. Operational traffic requires permission, minimal sensitive information and secure storage (Jobin et al., 2019).

Conclusion

The project demonstrates that classification performance can be obtained with tree-based models on this dataset. Random Forest has the observed final test macro F1, while Decision Tree remains the validation-selected recommendation. The neural network performs strongly but shows weaker recall for some smaller categories, and Naive Bayes exposes the limitations of relying on overall accuracy. The learning is that preprocessing, controlled selection and class-level interpretation are as important as choosing a complex algorithm. These findings should guide further evaluation while remaining bounded by the dataset, partition and provenance limitations documented above. These sources support methodological explanations; all numerical comparisons remain specific to the supplied notebook and its saved experimental outputs.

References

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Domingos, P., & Pazzani, M. (1997). On the optimality of the simple Bayesian classifier under zero-one loss. *Machine Learning, 29*(2–3), 103–130. https://doi.org/10.1023/A:1007413511361

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer. https://doi.org/10.1007/978-0-387-84858-7

He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering, 21*(9), 1263–1284. https://doi.org/10.1109/TKDE.2008.239

Jobin, A., Ienca, M., & Vayena, E. (2019). The global landscape of AI ethics guidelines. *Nature Machine Intelligence, 1*(9), 389–399. https://doi.org/10.1038/s42256-019-0088-2

Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data, 6*(4), Article 15. https://doi.org/10.1145/2382577.2382579

Kingma, D. P., & Ba, J. (2015). *Adam: A method for stochastic optimization*. International Conference on Learning Representations. https://arxiv.org/abs/1412.6980

Prechelt, L. (1998). Early stopping—but when? In G. B. Orr & K.-R. Müller (Eds.), *Neural networks: Tricks of the trade* (pp. 55–69). Springer. https://doi.org/10.1007/3-540-49430-8_3

Sharmila, B. S., & Nagapadma, R. (2023). Quantized autoencoder (QAE) intrusion detection system for anomaly detection in resource-constrained IoT devices using RT-IoT2022 dataset. *Cybersecurity, 6*, Article 41. https://doi.org/10.1186/s42400-023-00178-5

Sokolova, M., & Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. *Information Processing & Management, 45*(4), 427–437. https://doi.org/10.1016/j.ipm.2009.03.002

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research, 15*, 1929–1958. https://jmlr.org/papers/v15/srivastava14a.html

Strobl, C., Boulesteix, A.-L., Zeileis, A., & Hothorn, T. (2007). Bias in random forest variable importance measures: Illustrations, sources and a solution. *BMC Bioinformatics, 8*, Article 25. https://doi.org/10.1186/1471-2105-8-25

Data Details

Supplied dataset: data/dataset.csv. Raw records: 123,117. Exact duplicates removed: 5,195. Deduplicated records: 117,922. Predictors: 83. Target classes: 12.

Supplied CSV SHA-256: 4abec3353b994d3e369996024095c64b18778e23b6d42e1302ac2cfce74825c4

Notebook saved CSV SHA-256: 956956c09c1764584fa08acd0f6876475626bcedcd6a6b1f8c492c2e9a2089ea. This hash is reproduced exactly when the supplied CSV uses Windows CRLF line endings; its supplied form uses LF line endings.

Recorded experiment seed: 42. Fitting: 75,469; validation: 18,868; test: 23,585; final development refit: 94,337.

Recorded historical versions: Python 3.12.10; NumPy 2.5.3; pandas 3.0.6; scikit-learn 1.9.1; TensorFlow 2.21.0. These are transcribed from saved notebook output, not independently verified environment specifications.

Evidence files supplied: Model_Comparison.ipynb and data/dataset.csv. Eight embedded figures were extracted without modification. Separate trained model artifacts and exported predictions were not present in the source ZIP.

Reporting verification: dataset counts and class supports were independently checked; the complete notebook was not rerun. This document corrects the inconsistent totals in Assessment 3.